Programming Cheat Sheet

2026

Carlos E. R. Oliveira • Kaique A. de Oliveira • Mateus A. Mizael

Sumário

1 STL **2**
1.1 sort . . . 2
1.2 lower_bound / upper_bound . . . 2
1.3 count / find . . . 2
1.4 next_permutation . . . 2
1.5 binary_search . . . 2
2 Matemática **2**
2.1 MDC / GCD . . . 2
2.2 MMC / LCM . . . 3
2.3 Fast Pow . . . 3
2.4 Combinatória . . . 3
2.5 Permutação . . . 4
2.6 Inverso Modular . . . 4
2.7 Totient de Euler . . . 4
2.8 Primos . . . 5
2.8.1 Crivo de Eratóstenes . . . 5
2.8.2 Fatoração Prima . . . 5
2.8.3 Contagem de Divisores . . . 5
3 Geometria **6**
3.1 Interseção de Segmentos . . . 6
4 Arrays **7**
4.1 Prefix Sum 2D . . . 7
4.2 Array de Diferenças 2D . . . 7
4.3 Sliding Window com Frequência . . . 7
4.4 Kadane . . . 8
4.5 Contagem de Subarrays com Soma K . . . 8
4.6 Contagem de Inversões . . . 8
5 Estruturas de Dados **9**
5.1 Fenwick Tree . . . 9
5.2 Segment Tree . . . 10
5.3 Lazy Segment Tree . . . 11
5.4 Merge Sort Tree . . . 12
5.5 DSU . . . 13
6 Grafos **14**
6.1 BFS 0-1 . . . 14
6.2 BFS Multi-Estado . . . 14
6.3 Dijkstra Multi-Estado . . . 15
6.4 MST — Kruskal . . . 16
6.5 MST — Prim . . . 17
6.6 Ordenação Topológica — Kahn . . . 17
6.7 Diâmetro de uma Árvore . . . 18
6.8 Grafo Bipartido . . . 19
6.9 Bellman-Ford . . . 20
6.10 Floyd-Warshall . . . 20
7 Programação Dinâmica **21**
7.1 Knapsack 0-1 . . . 21
7.2 Knapsack Inteira . . . 21
7.3 LIS — Longest Increasing Subsequence . . . 22
7.4 LCS — Longest Common Subsequence . . . 22
7.5 Maximum Sum 2D . . . 23
7.6 PD de Intervalos . . . 23

1. STL

1.1 sort

Complexidade: $O(N \log N)$.

```
sort(a.begin(), a.end());                 // crescente
sort(a.rbegin(), a.rend());               // decrescente
sort(a.begin(), a.end(), greater<int>()); // decrescente
```

1.2 lower_bound / upper_bound

- `lower_bound`: primeiro elemento $\geq x$
- `upper_bound`: primeiro elemento $> x$

Complexidade: $O(\log N)$.

```
int lb = lower_bound(a.begin(), a.end(), x) - a.begin();
int ub = upper_bound(a.begin(), a.end(), x) - a.begin();
```

Para descobrir a quantidade de ocorrências de x:

```
int cnt = upper_bound(a.begin(), a.end(), x) - lower_bound(a.begin(), a.end(), x);
```

1.3 count / find

Operam sobre o intervalo $[first, last)$.
`count` retorna a quantidade de ocorrências.
`find` retorna um iterador para a primeira ocorrência ou `end()` caso não encontre.
Complexidade: $O(N)$.

```
int cnt = count(a.begin(), a.end(), x);
auto it = find(a.begin(), a.end(), x);

if (it != a.end()) {
    // encontrado
}
```

1.4 next_permutation

Transforma o intervalo na próxima permutação lexicográfica.
Retorna `false` se o intervalo já estiver na última permutação e o transforma na primeira.
Complexidade: $O(N)$.

```
sort(a.begin(), a.end());

do {
    // usa a permutacao atual
} while (next_permutation(a.begin(), a.end()));
```

1.5 binary_search

Verifica se x existe no intervalo ordenado e retorna `true` ou `false`.
Complexidade: $O(\log N)$.

```
bool exists = binary_search(a.begin(), a.end(), x);
```

2. Matemática

2.1 MDC / GCD

Algoritmo de Euclides.
STL: `std::gcd(a, b)`
Complexidade: $O(\log(\min(a, b)))$.

```
long long gcd(long long a, long long b) {
    while (b) {
        a %= b;
        swap(a, b);
    }

    return a;
}
```

2.2 MMC / LCM

$$\text{lcm}(a, b) = \frac{a}{\text{gcd}(a, b)} \cdot b$$

STL: `std::lcm(a, b)`
Complexidade: $O(\log(\min(a, b)))$.

```
long long lcm(long long a, long long b) {
    return a / gcd(a, b) * b;
}
```

2.3 Fast Pow

Calcula a^b usando exponenciação binária.
Complexidade: $O(\log b)$.

```
long long binpow(long long a, long long b) {
    long long res = 1;

    while (b) {
        if (b & 1)
            res *= a;

        a *= a;
        b >>= 1;
    }

    return res;
}
```

2.4 Combinatória

Combinações distintas escolhendo k elementos dentre n elementos, sem importar a ordem:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Propriedade útil:

$$\binom{n}{k} = \binom{n}{n-k}$$

Para valores pequenos, pode ser calculado iterativamente evitando calcular os fatoriais completos.

```
long long C(int n, int k) {
    if (k > n - k)
        k = n - k;

    long long res = 1;

    for (int i = 1; i <= k; i++)
        res = res * (n - k + i) / i;

    return res;
}
```

2.5 Permutação

Número de maneiras de escolher e ordenar k elementos dentre n:

$$P(n,k) = \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+1)$$

Complexidade: $O(k)$.

```
long long P(int n, int k) {
    long long res = 1;

    for (int i = 0; i < k; i++)
        res *= n - i;

    return res;
}
```

2.6 Inverso Modular

x é o inverso modular de a módulo m se:

$$ax \equiv 1 \pmod{m}$$

Existe se e somente se:

$$\gcd(a,m) = 1$$

Se m é primo:

$$a^{-1} \equiv a^{m-2} \pmod{m}$$

Complexidade: $O(\log m)$.

```
long long modInverse(long long a, long long mod) {
    return binpow(a, mod - 2, mod);
}
```

Requer: m primo e $a \not\equiv 0 \pmod{m}$.

2.7 Totient de Euler

$\varphi(n)$ = quantidade de inteiros em $[1,n]$ coprimos com n.

Se:

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

então:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

Complexidade: $O(\sqrt{n})$.

```
long long phi(long long n) {
    long long res = n;

    for (long long p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0)
                n /= p;

            res -= res / p;
        }
    }

    if (n > 1)
        res -= res / n;

    return res;
}
```

2.8 Primos

2.8.1 Crivo de Eratóstenes

Encontra todos os primos até N.

Complexidade: $O(N \log \log N)$.

```
vector<bool> isPrime(N + 1, true);
isPrime[0] = isPrime[1] = false;

for (int i = 2; i * i <= N; i++) {
    if (!isPrime[i])
        continue;

    // j = i * i eh uma otimizacao,
    // tenha em mente que em alguns casos pode ser necessario iniciar com i + i
    for (int j = i * i; j <= N; j += i)
        isPrime[j] = false;
}
```

2.8.2 Fatoração Prima

Decompõe n em seus fatores primos.

Complexidade: $O(\sqrt{n})$.

```
vector<pair<long long, int>> factorize(long long n) {
    vector<pair<long long, int>> factors;

    for (long long p = 2; p * p <= n; p++) {
        if (n % p != 0)
            continue;

        int cnt = 0;

        while (n % p == 0) {
            n /= p;
            cnt++;
        }

        factors.push_back({p, cnt});
    }

    if (n > 1)
        factors.push_back({n, 1});

    return factors;
}
```

2.8.3 Contagem de Divisores

Se:

$$n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$$

então a quantidade de divisores positivos de n é:

$$d(n) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

Complexidade: $O(\sqrt{n})$.

```
long long divisorCount(long long n) {
    long long res = 1;

    for (long long p = 2; p * p <= n; p++) {
        if (n % p != 0)
            continue;

        int cnt = 0;

        while (n % p == 0) {
```

```
            n /= p;
            cnt++;
        }

        res *= cnt + 1;
    }

    if (n > 1)
        res *= 2;

    return res;
}
```

3. Geometria

3.1 Interseção de Segmentos

Dados dois segmentos $\overline{AB}$ e $\overline{CD}$, verifica se eles possuem pelo menos um ponto em comum.

Defina a orientação:

$$cross(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x)$$

> 0: C está a esquerda de AB
< 0: C está a direita de AB
$= 0$: pontos colineares

Dois segmentos próprios se intersectam quando:

$$orient(A, B, C) \neq orient(A, B, D)$$

e

$$orient(C, D, A) \neq orient(C, D, B)$$

Os casos colineares exigem verificação adicional.

Complexidade: $O(1)$.

```
struct Point {
    long long x, y;
};

long long cross(const Point& a, const Point& b, const Point& c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

bool onSegment(const Point& a, const Point& b, const Point& point) {
    return cross(a, b, point) == 0 &&
           min(a.x, b.x) <= point.x &&
           point.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= point.y &&
           point.y <= max(a.y, b.y);
}

bool segmentsIntersect(const Point& a, const Point& b, const Point& c, const Point& d) {
    long long orientation1 = cross(a, b, c);
    long long orientation2 = cross(a, b, d);
    long long orientation3 = cross(c, d, a);
    long long orientation4 = cross(c, d, b);

    // Caso geral: os segmentos se cruzam propriamente.
    if ((orientation1 > 0) != (orientation2 > 0) &&
        (orientation3 > 0) != (orientation4 > 0)) {
        return true;
    }

    // Casos degenerados: um ponto esta sobre o outro segmento.
    if (orientation1 == 0 &&
        onSegment(a, b, c))
        return true;

    if (orientation2 == 0 &&
```

```
        onSegment(a, b, d))
        return true;

    if (orientation3 == 0 &&
        onSegment(c, d, a))
        return true;

    if (orientation4 == 0 &&
        onSegment(c, d, b))
        return true;

    return false;
}
```

4. Arrays

4.1 Prefix Sum 2D

Permite consultar a soma de qualquer submatriz em $O(1)$ após pré-processamento.

Pré-processamento: $O(NM)$.

Consulta: $O(1)$.

$$prefix[i][j] = a[i][j] + prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1]$$

Para a submatriz (r_1, c_1) até (r_2, c_2):

$$sum = prefix[r_2][c_2] - prefix[r_1-1][c_2] - prefix[r_2][c_1-1] + prefix[r_1-1][c_1-1]$$

Usando índices de 1 a N, M:

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        prefix[i][j] = a[i][j] + prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1];

int sum = prefix[r2][c2] - prefix[r1-1][c2] - prefix[r2][c1-1] + prefix[r1-1][c1-1];
```

4.2 Array de Diferenças 2D

Permite aplicar incrementos em submatrizes em $O(1)$ cada. O resultado final é obtido com uma soma de prefixos 2D.

Para adicionar x à região $[r_1, r_2] \times [c_1, c_2]$:

```
diff[r1][c1] += x;
diff[r2+1][c1] -= x;
diff[r1][c2+1] -= x;
diff[r2+1][c2+1] += x;
```

Depois, reconstrua o array:

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        diff[i][j] += diff[i-1][j] + diff[i][j-1] - diff[i-1][j-1];
```

Atualização: $O(1)$.

Reconstrução: $O(NM)$.

4.3 Sliding Window com Frequência

Mantém uma janela $[l, r]$ e suas frequências.

Padrão para encontrar uma janela que satisfaz uma condição monotônica.

Complexidade: $O(N)$.

```
int l = 0;
int have = 0;

for (int r = 0; r < n; r++) {
    freq[a[r]]++;

    if (freq[a[r]] == 1)
        have++;
```

```
    while (have == need) {
        // [l, r] eh uma janela valida

        freq[a[l]]--;

        if (freq[a[l]] == 0)
            have--;

        l++;
    }
}
```

Para contar subarrays válidas, frequentemente:

```
ans += r - l + 1;
```

4.4 Kadane

Encontra a maior soma de uma subarray não vazia.

Complexidade: $O(N)$.

```
long long cur = 0;
long long ans = LLONG_MIN;

for (long long x : a) {
    cur = max(x, cur + x);
    ans = max(ans, cur);
}
```

4.5 Contagem de Subarrays com Soma K

Conta a quantidade de subarrays contínuas cuja soma é exatamente K.

Mantém a frequência dos prefixos já encontrados.

Complexidade: $O(N)$ esperado.

```
long long countSubarrays(const vector<int>& a, long long k) {
    unordered_map<long long, int> freq;

    freq[0] = 1;

    long long sum = 0;
    long long ans = 0;

    for (long long x : a) {
        sum += x;

        if (freq.count(sum - k))
            ans += freq[sum - k];

        freq[sum]++;
    }

    return ans;
}
```

4.6 Contagem de Inversões

Um par (i, j) é uma inversão se:

$$i < j \quad \text{e} \quad a_i > a_j$$

Pode ser calculado durante o Merge Sort.

Complexidade: $O(N \log N)$.

```
long long merge(vector<int>& a, int l, int m, int r) {
    vector<int> tmp;
    long long inv = 0;
```

```
    int i = l, j = m + 1;

    while (i <= m && j <= r) {
        if (a[i] <= a[j]) {
            tmp.push_back(a[i++]);
        } else {
            tmp.push_back(a[j++]);
            inv += m - i + 1;
        }
    }

    while (i <= m)
        tmp.push_back(a[i++]);

    while (j <= r)
        tmp.push_back(a[j++]);

    copy(tmp.begin(), tmp.end(), a.begin() + l);

    return inv;
}

long long mergeSort(vector<int>& a, int l, int r) {
    if (l >= r)
        return 0;

    int m = (l + r) / 2;

    long long inv = 0;

    inv += mergeSort(a, l, m);
    inv += mergeSort(a, m + 1, r);
    inv += merge(a, l, m, r);

    return inv;
}
```

Uso:

```
long long inversions = mergeSort(a, 0, n - 1);
```

5. Estruturas de Dados

5.1 Fenwick Tree

Suporta soma de prefixo e atualização pontual.
Atualização: $O(\log N)$
Consulta de prefixo: $O(\log N)$
Soma em $[l, r]$**:** $O(\log N)$
Usa índices de 1 a N.

```
struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int n) : n(n), bit(n + 1) {}

    void add(int i, long long x) {
        for (; i <= n; i += i & -i)
            bit[i] += x;
    }

    long long sum(int i) const {
        long long res = 0;

        for (; i > 0; i -= i & -i)
            res += bit[i];
```

```
        return res;
    }

    long long query(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};
```

5.2 Segment Tree

Suporta consultas em intervalos e atualizações pontuais.
Construção: $O(N)$.
Consulta: $O(\log N)$.
Atualização pontual: $O(\log N)$.

```
struct SegTree {
    int n;
    vector<long long> tree;

    SegTree(const vector<long long>& a) {
        n = a.size();
        tree.resize(4 * n);
        build(a, 1, 0, n - 1);
    }

    void build(const vector<long long>& a, int p, int l, int r) {
        if (l == r) {
            tree[p] = a[l];
            return;
        }

        int m = (l + r) / 2;

        build(a, p * 2, l, m);
        build(a, p * 2 + 1, m + 1, r);

        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    void update(int idx, long long val, int p, int l, int r) {
        if (l == r) {
            tree[p] = val;
            return;
        }

        int m = (l + r) / 2;

        if (idx <= m)
            update(idx, val, p * 2, l, m);
        else
            update(idx, val, p * 2 + 1, m + 1, r);

        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    long long query(int ql, int qr, int p, int l, int r) {
        if (qr < l || r < ql)
            return 0;

        if (ql <= l && r <= qr)
            return tree[p];

        int m = (l + r) / 2;

        return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m + 1, r);
    }
};
```

Uso:

```
SegTree st(a);

st.update(idx, val, 1, 0, n - 1);

long long ans = st.query(l, r, 1, 0, n - 1);
```

5.3 Lazy Segment Tree

Permite atualizações em intervalos sem visitar cada elemento.
Construção: $O(N)$.
Consulta: $O(\log N)$.
Atualização em intervalo: $O(\log N)$.

```
struct LazySegTree {
    int n;
    vector<long long> tree, lazy;

    LazySegTree(const vector<long long>& a) {
        n = a.size();
        tree.resize(4 * n);
        lazy.resize(4 * n);
        build(a, 1, 0, n - 1);
    }

    void build(const vector<long long>& a, int p, int l, int r) {
        if (l == r) {
            tree[p] = a[l];
            return;
        }

        int m = (l + r) / 2;

        build(a, p * 2, l, m);
        build(a, p * 2 + 1, m + 1, r);

        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    void apply(int p, int l, int r, long long value) {
        tree[p] += value * (r - l + 1);
        lazy[p] += value;
    }

    void push(int p, int l, int r) {
        if (lazy[p] == 0 || l == r)
            return;

        int m = (l + r) / 2;

        // Propaga a atualizacao pendente para os filhos.
        apply(p * 2, l, m, lazy[p]);
        apply(p * 2 + 1, m + 1, r, lazy[p]);

        lazy[p] = 0;
    }

    void update(int ql, int qr, long long value,
                int p, int l, int r) {
        if (qr < l || r < ql)
            return;

        if (ql <= l && r <= qr) {
            apply(p, l, r, value);
            return;
        }
```

```
        push(p, l, r);

        int m = (l + r) / 2;

        update(ql, qr, value, p * 2, l, m);
        update(ql, qr, value, p * 2 + 1, m + 1, r);

        tree[p] = tree[p * 2] + tree[p * 2 + 1];
    }

    long long query(int ql, int qr, int p, int l, int r) {
        if (qr < l || r < ql)
            return 0;

        if (ql <= l && r <= qr)
            return tree[p];

        push(p, l, r);

        int m = (l + r) / 2;

        return query(ql, qr, p * 2, l, m) + query(ql, qr, p * 2 + 1, m + 1, r);
    }
};
```

Uso:

```
LazySegTree st(a);

st.update(l, r, value, 1, 0, n - 1);

long long ans = st.query(l, r, 1, 0, n - 1);
```

5.4 Merge Sort Tree

Segment Tree onde cada nó mantém os elementos do seu intervalo ordenados.

Permite consultas de ordem em intervalos usando `lower_bound` / `upper_bound`.

Construção: $O(N \log N)$.

Consulta: $O(\log^2 N)$.

```
struct MergeSortTree {
    int n;
    vector<vector<int>> tree;

    MergeSortTree(const vector<int>& a) {
        n = a.size();
        tree.resize(4 * n);
        build(a, 1, 0, n - 1);
    }

    void build(const vector<int>& a, int p, int l, int r) {
        if (l == r) {
            tree[p] = {a[l]};
            return;
        }

        int m = (l + r) / 2;

        build(a, p * 2, l, m);
        build(a, p * 2 + 1, m + 1, r);

        // Cada no armazena os elementos do intervalo ordenados.
        merge(tree[p * 2].begin(),
              tree[p * 2].end(),
              tree[p * 2 + 1].begin(),
              tree[p * 2 + 1].end(),
              back_inserter(tree[p]));
    }
```

```
    int query(int ql, int qr, int x, int p, int l, int r) {
        // Intervalo atual nao participa da consulta.
        if (qr < l || r < ql)
            return 0;

        if (ql <= l && r <= qr)
            return lower_bound(tree[p].begin(), tree[p].end(), x) - tree[p].begin();

        int m = (l + r) / 2;

        return query(ql, qr, x, p * 2, l, m) + query(ql, qr, x, p * 2 + 1, m + 1, r);
    }
};
```

O exemplo acima conta elementos $< x$ no intervalo $[l, r]$.
Para contar elementos $\leq x$, use `upper_bound`.

5.5 DSU

Disjoint Set Union / Union-Find.
Mantém componentes disjuntos e suporta união e consulta de componentes.
Complexidade com Compressão de Caminho: $O(\alpha(N))$ por operação.
Complexidade sem Compressão de Caminho: $O(log(N))$ por operação.

```
struct DSU {
    vector<int> parent;
    vector<int> componentSize;

    DSU(int vertexCount) : parent(vertexCount), componentSize(vertexCount, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    // Sem compressao de caminho.
    int find(int vertex) {
        // Percorre a arvore ate encontrar a raiz.
        while (vertex != parent[vertex]) {
            vertex = parent[vertex];
        }

        return vertex;
    }

    // Com compresao de caminho.
    int findCompressed(int vertex) {
        int root = vertex;

        // Primeiro encontra a raiz do conjunto.
        while (root != parent[root]) {
            root = parent[root];
        }

        // Faz todos os vertices do caminho apontarem para a raiz.
        while (vertex != root) {
            int nextVertex = parent[vertex];
            parent[vertex] = root;
            vertex = nextVertex;
        }

        return root;
    }

    bool unite(int firstVertex, int secondVertex) {
        int firstRoot = findCompressed(firstVertex);
        int secondRoot = findCompressed(secondVertex);

        if (firstRoot == secondRoot)
            return false;
```

```
        // Mantem a maior arvore como raiz.
        if (componentSize[firstRoot] < componentSize[secondRoot])
            swap(firstRoot, secondRoot);

        parent[secondRoot] = firstRoot;
        componentSize[firstRoot] += componentSize[secondRoot];

        return true;
    }
};
```

6. Grafos

6.1 BFS 0-1

Usado em grafos cujas arestas possuem peso apenas 0 ou 1.

Mantém os vértices em uma deque:

- peso 0: adiciona no início;
- peso 1: adiciona no final.

Complexidade: $O(V + E)$.

```
struct Edge {
    int destination;
    int weight;
};

vector<long long> zeroOneBFS(const vector<vector<Edge>>& graph, int origin) {
    int vertexCount = graph.size();

    vector<long long> distance(vertexCount, INF);

    deque<int> vertices;

    distance[origin] = 0;
    vertices.push_front(origin);

    while (!vertices.empty()) {
        int currentVertex = vertices.front();
        vertices.pop_front();

        for (const Edge& edge : graph[currentVertex]) {
            long long newDistance = distance[currentVertex] + edge.weight;

            if (newDistance >= distance[edge.destination])
                continue;

            distance[edge.destination] = newDistance;

            // Arestas de peso 0 devem ser processadas antes das arestas de peso 1.
            if (edge.weight == 0)
                vertices.push_front(edge.destination);
            else
                vertices.push_back(edge.destination);
        }
    }

    return distance;
}
```

6.2 BFS Multi-Estado

Usado quando chegar ao mesmo vértice com estados diferentes produz situações diferentes.

O princípio é tratar $(v, state)$ como um vértice diferente.

Complexidade: $O(VS + ES)$, onde S é o número de estados.

```cpp
struct State {
    int vertex;
    int state;
};

vector<vector<int>> multiStateBFS(const vector<vector<int>>& graph, int origin, int stateCount) {
    int vertexCount = graph.size();

    vector<vector<int>> distance(vertexCount, vector<int>(stateCount, -1));

    queue<State> states;

    // Estado inicial.
    distance[origin][0] = 0;
    states.push({origin, 0});

    while (!states.empty()) {
        State current = states.front();
        states.pop();

        int currentDistance = distance[current.vertex][current.state];

        for (int neighbor : graph[current.vertex]) {
            // Exemplo: transicao que mantem o estado.
            int nextState = current.state;

            if (distance[neighbor][nextState] != -1)
                continue;

            distance[neighbor][nextState] = currentDistance + 1;

            states.push({neighbor, nextState});
        }
    }

    return distance;
}
```

O trecho que determina `nextState` deve ser adaptado ao problema.
A dimensão do vetor `distance` representa o espaço de estados.

6.3 Dijkstra Multi-Estado

Extensão do Dijkstra em que o vértice do grafo é combinado com um estado adicional.
O estado completo é $(v, state)$.
Complexidade: $O((VS + ES)\log(VS))$.

```cpp
struct Edge {
    int destination;
    long long weight;
};

struct State {
    long long distance;
    int vertex;
    int state;

    bool operator>(const State& other) const {
        return distance > other.distance;
    }
};

vector<vector<long long>> multiDijkstra(const vector<vector<Edge>>& graph, int origin, int stateCount) {
    int vertexCount = graph.size();

    vector<vector<long long>> distance(vertexCount, vector<long long>(stateCount, INF));

    priority_queue<
```

```
        State,
        vector<State>,
        greater<State>
    > priorityQueue;

    distance[origin][0] = 0;

    priorityQueue.push({0, origin, 0});

    while (!priorityQueue.empty()) {
        State current = priorityQueue.top();
        priorityQueue.pop();

        // Ignora entradas antigas da priority_queue.
        if (current.distance != distance[current.vertex][current.state])
            continue;

        for (const Edge& edge : graph[current.vertex]) {
            // Adapte esta transicao conforme o problema.
            int nextState = current.state;

            long long newDistance = current.distance + edge.weight;

            if (newDistance >= distance[edge.destination][nextState])
                continue;

            distance[edge.destination][nextState] = newDistance;

            priorityQueue.push({newDistance, edge.destination, nextState});
        }
    }

    return distance;
}
```

O ponto mais importante é lembrar que:

$$\boxed{\text{vértice real} \rightarrow (v, state)}$$

Cada combinação é tratada como um vértice independente.

6.4 MST — Kruskal

Encontra uma Minimum Spanning Tree usando as arestas ordenadas por peso.
Usa DSU para evitar ciclos.
Complexidade: $O(E \log E)$.

```
struct Edge {
    int firstVertex;
    int secondVertex;
    long long weight;
};

long long kruskal(int vertexCount, vector<Edge> edges) {
    sort(edges.begin(), edges.end(),
        [](const Edge& first, const Edge& second) {
            return first.weight < second.weight;
        }
    );

    DSU disjointSet(vertexCount);

    long long totalWeight = 0;
    int selectedEdges = 0;

    for (const Edge& edge : edges) {
        // Se os vertices ja estao conectados, adicionar esta aresta criaria um ciclo.
        if (!disjointSet.unite(edge.firstVertex, edge.secondVertex))
            continue;
```

```
        totalWeight += edge.weight;
        selectedEdges++;

        if (selectedEdges == vertexCount - 1)
            break;
    }

    // Se nao houver arvore geradora, o grafo nao eh conexo.
    if (selectedEdges != vertexCount - 1)
        return -1;

    return totalWeight;
}
```

6.5 MST — Prim

Adequado para grafos densos representados por matriz de adjacência.

A cada iteração, adiciona o vértice ainda não utilizado com a menor aresta conhecida para a MST.

Complexidade: $O(V^2)$.

```
long long prim(const vector<vector<long long>>& graph) {
    int vertexCount = graph.size();

    vector<long long> minimumEdge(vertexCount, INF);

    vector<bool> used(vertexCount, false);

    long long totalWeight = 0;

    // Comeca a MST pelo vertice 0.
    minimumEdge[0] = 0;

    for (int iteration = 0; iteration < vertexCount; iteration++) {
        int currentVertex = -1;

        // Escolhe o vertice nao utilizado com a menor aresta para a MST.
        for (int vertex = 0; vertex < vertexCount; vertex++)
            if (!used[vertex] && (currentVertex == -1 ||
                minimumEdge[vertex] < minimumEdge[currentVertex]))
                currentVertex = vertex;

        // O grafo nao eh conexo.
        if (currentVertex == -1 || minimumEdge[currentVertex] == INF)
            return -1;

        used[currentVertex] = true;
        totalWeight += minimumEdge[currentVertex];

        // Atualiza a menor aresta que conecta cada vertice restante a MST.
        for (int neighbor = 0; neighbor < vertexCount; neighbor++)
            if (!used[neighbor] && graph[currentVertex][neighbor] < minimumEdge[neighbor])
                minimumEdge[neighbor] = graph[currentVertex][neighbor];
    }

    return totalWeight;
}
```

6.6 Ordenação Topológica — Kahn

Ordena os vértices de um DAG de forma que, para toda aresta $u \to v$, u apareça antes de v utilizando os graus de entrada (*in-degree*).

Complexidade: $O(V + E)$.

```
vector<int> topologicalSort(const vector<vector<int>>& graph) {
    int vertexCount = graph.size();
```

```
    vector<int> inDegree(vertexCount, 0);

    for (int vertex = 0; vertex < vertexCount; vertex++)
        for (int neighbor : graph[vertex])
            inDegree[neighbor]++;

    queue<int> readyVertices;

    // Vertices sem dependencias podem ser processados imediatamente.
    for (int vertex = 0; vertex < vertexCount; vertex++)
        if (inDegree[vertex] == 0)
            readyVertices.push(vertex);

    vector<int> order;

    while (!readyVertices.empty()) {
        int currentVertex = readyVertices.front();
        readyVertices.pop();

        order.push_back(currentVertex);

        for (int neighbor : graph[currentVertex]) {
            inDegree[neighbor]--;

            // Todas as dependencias de neighbor foram processadas.
            if (inDegree[neighbor] == 0)
                readyVertices.push(neighbor);
        }
    }

    // Se nem todos os vertices foram processados, existe um ciclo.
    if ((int)order.size() != vertexCount)
        return {};

    return order;
}
```

6.7 Diâmetro de uma Árvore

O diâmetro é o maior caminho entre dois vértices da árvore.
Em uma árvore não ponderada:

1. Faça BFS/DFS a partir de qualquer vértice.
2. Pegue o vértice mais distante A.
3. Faça outra BFS/DFS a partir de A.
4. A maior distância encontrada é o diâmetro.

Complexidade: $O(V)$.

```
pair<int, int> bfsFarthest(const vector<vector<int>>& tree, int origin) {
    int vertexCount = tree.size();

    vector<int> distance(vertexCount, -1);

    queue<int> vertices;

    distance[origin] = 0;
    vertices.push(origin);

    int farthestVertex = origin;

    while (!vertices.empty()) {
        int currentVertex = vertices.front();
        vertices.pop();

        if (distance[currentVertex] > distance[farthestVertex])
```

```
            farthestVertex = currentVertex;

        for (int neighbor : tree[currentVertex]) {
            if (distance[neighbor] != -1)
                continue;

            distance[neighbor] = distance[currentVertex] + 1;

            vertices.push(neighbor);
        }
    }

    return {farthestVertex, distance[farthestVertex]};
}

int treeDiameter(const vector<vector<int>>& tree) {
    // O vertice mais distante de qualquer vertice eh uma extremidade do diametro.
    auto [endpoint, unusedDistance] = bfsFarthest(tree, 0);

    // A maior distancia a partir dessa extremidade eh exatamente o diametro.
    auto [otherEndpoint, diameter] = bfsFarthest(tree, endpoint);

    return diameter;
}
```

6.8 Grafo Bipartido

Um grafo é bipartido se seus vértices podem ser divididos em dois conjuntos, de modo que nenhuma aresta conecte vértices do mesmo conjunto.

Pode ser verificado colorindo os vértices durante uma BFS/DFS.

Complexidade: $O(V + E)$.

```
bool isBipartite(const vector<vector<int>>& graph) {
    int vertexCount = graph.size();

    // -1  = nao colorido
    // 0/1 = duas cores possiveis
    vector<int> color(vertexCount, -1);

    queue<int> vertices;

    for (int startVertex = 0; startVertex < vertexCount; startVertex++) {
        if (color[startVertex] != -1)
            continue;

        color[startVertex] = 0;
        vertices.push(startVertex);

        while (!vertices.empty()) {
            int currentVertex = vertices.front();
            vertices.pop();

            for (int neighbor : graph[currentVertex]) {
                if (color[neighbor] == -1) {
                    // Vertices adjacentes precisam receber cores diferentes.
                    color[neighbor] = color[currentVertex] ^ 1;

                    vertices.push(neighbor);
                }
                else if (color[neighbor] == color[currentVertex]) {
                    // Uma aresta entre vertices da mesma cor prova que nao eh bipartido.
                    return false;
                }
            }
        }
    }

    return true;
```

```
}
```

6.9 Bellman-Ford

Encontra menores caminhos a partir de uma origem em grafos com pesos negativos.
Também detecta ciclos negativos alcançáveis a partir da origem.
Complexidade: $O(VE)$.

```
struct Edge {
    int origin;
    int destination;
    long long weight;
};

pair<vector<long long>, bool> bellmanFord(int vertexCount, const vector<Edge>& edges, int origin) {
    vector<long long> distance(vertexCount, INF);

    distance[origin] = 0;

    // O menor caminho simples possui no maximo V - 1 arestas.
    for (int iteration = 0; iteration < vertexCount - 1; iteration++) {
        bool changed = false;

        for (const Edge& edge : edges) {
            if (distance[edge.origin] == INF)
                continue;

            long long newDistance = distance[edge.origin] + edge.weight;

            if (newDistance < distance[edge.destination]) {
                distance[edge.destination] = newDistance;
                changed = true;
            }
        }

        // Nenhuma distancia mudou: ja convergiu.
        if (!changed)
            break;
    }

    // Se ainda for possivel relaxar uma aresta, existe um ciclo negativo alcancavel.
    for (const Edge& edge : edges) {
        if (distance[edge.origin] == INF)
            continue;

        if (distance[edge.origin] + edge.weight < distance[edge.destination])
            return {distance, true};
    }

    return {distance, false};
}
```

6.10 Floyd-Warshall

Encontra os menores caminhos entre **todos os pares** de vértices.
Também funciona com arestas negativas, desde que não existam ciclos negativos.
Complexidade: $O(V^3)$.
Memória: $O(V^2)$.

```
void floydWarshall(vector<vector<long long>>& distance) {
    int vertexCount = distance.size();

    for (int intermediate = 0; intermediate < vertexCount; intermediate++) {
        for (int origin = 0; origin < vertexCount; origin++) {
            if (distance[origin][intermediate] == INF)
                continue;
```

```
            for (int destination = 0; destination < vertexCount; destination++) {
                if (distance[intermediate][destination] == INF)
                    continue;

                distance[origin][destination] =
                    min(distance[origin][destination],
                        distance[origin][intermediate] + distance[intermediate][destination]);
            }
        }
    }
}
```

Inicialização:

```
vector<vector<long long>> distance(vertexCount, vector<long long>(vertexCount, INF));

for (int vertex = 0; vertex < vertexCount; vertex++)
    distance[vertex][vertex] = 0;

for (const Edge& edge : edges) {
    distance[edge.origin][edge.destination] =
        min(distance[edge.origin][edge.destination], edge.weight);
}
```

Após o algoritmo, se:

$$distance[v][v] < 0$$

para algum vértice v, existe um ciclo negativo.

7. Programação Dinâmica

7.1 Knapsack 0-1

Cada item pode ser escolhido no máximo uma vez.

Complexidade: $O(NC)$.

```
int knapsack01(const vector<int>& weight, const vector<int>& value, int capacity) {
    int itemCount = weight.size();

    vector<long long> dp(capacity + 1, 0);

    for (int item = 0; item < itemCount; item++) {
        // Percorrer de tras para frente impede que o mesmo item seja usado mais de uma vez.
        for (int currentCapacity = capacity; currentCapacity >= weight[item]; currentCapacity--)
            dp[currentCapacity] =
                max(dp[currentCapacity],
                    dp[currentCapacity - weight[item]] + value[item]);
    }

    return dp[capacity];
}
```

A transicao é:

$$dp[c] = \max(dp[c], dp[c - w_i] + v_i)$$

A iteração da capacidade deve ser **decrescente**.

7.2 Knapsack Inteira

Cada item pode ser escolhido qualquer número de vezes.

Complexidade: $O(NC)$.

```
int unboundedKnapsack(const vector<int>& weight, const vector<int>& value, int capacity) {
    int itemCount = weight.size();

    vector<long long> dp(capacity + 1, 0);

    for (int item = 0; item < itemCount; item++) {
        // Percorrer para frente permite reutilizar o mesmo item varias vezes.
        for (int currentCapacity = weight[item]; currentCapacity <= capacity; currentCapacity++)
            dp[currentCapacity] =
```

```
                max(dp[currentCapacity],
                    dp[currentCapacity - weight[item]] + value[item]);
    }

    return dp[capacity];
}
```

A diferença essencial para o Knapsack 0-1 é a direção do loop:

0-1: decrescente	Inteira: crescente

7.3 LIS — Longest Increasing Subsequence

Encontra o comprimento da maior subsequência estritamente crescente.

Complexidade: $O(N \log N)$.

```
int longestIncreasingSubsequence(const vector<int>& values) {
    vector<int> tails;

    for (int value : values) {
        // Primeiro elemento >= value.
        auto position = lower_bound(tails.begin(), tails.end(), value);

        if (position == tails.end())
            tails.push_back(value);
        else
            *position = value;
    }

    return tails.size();
}
```

`tails[i]` representa o menor valor possível para o último elemento de uma LIS de tamanho $i + 1$.

Para subsequência **não decrescente**, use `upper_bound`.

7.4 LCS — Longest Common Subsequence

Encontra o tamanho da maior subsequência comum entre duas sequências.

Complexidade: $O(NM)$.

```
int longestCommonSubsequence(const string& first, const string& second) {
    int firstLength = first.size();
    int secondLength = second.size();

    vector<vector<int>> dp(firstLength + 1, vector<int>(secondLength + 1, 0));

    for (int firstIndex = 1; firstIndex <= firstLength; firstIndex++) {
        for (int secondIndex = 1; secondIndex <= secondLength; secondIndex++) {
            if (first[firstIndex - 1] == second[secondIndex - 1]) {
                // Os caracteres podem fazer parte da subsequencia comum.
                dp[firstIndex][secondIndex] = dp[firstIndex - 1][secondIndex - 1] + 1;
            } else {
                // Descarta um dos dois caracteres.
                dp[firstIndex][secondIndex] =
                    max(dp[firstIndex - 1][secondIndex],
                        dp[firstIndex][secondIndex - 1]);
            }
        }
    }

    return dp[firstLength][secondLength];
}
```

Transição:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & a_i = b_j \\ \max(dp[i-1][j], dp[i][j-1]), & a_i \neq b_j \end{cases}$$

7.5 Maximum Sum 2D

Encontra a submatriz com a maior soma.

Pode ser reduzido a várias execuções do Kadane.

Fixamos duas linhas e comprimimos as colunas em um vetor.

Complexidade: $O(N^2M)$.

Para uma matriz $N \times M$:

```
long long maximumSum2D(const vector<vector<long long>>& matrix) {
    int rowCount = matrix.size();
    int columnCount = matrix[0].size();

    long long answer = LLONG_MIN;

    for (int topRow = 0; topRow < rowCount; topRow++) {
        vector<long long> columnSum(columnCount, 0);

        for (int bottomRow = topRow; bottomRow < rowCount; bottomRow++) {
            // Soma cada coluna entre topRow e bottomRow.
            for (int column = 0; column < columnCount; column++)
                columnSum[column] += matrix[bottomRow][column];

            // Agora o problema virou Kadane 1D.
            long long currentSum = 0;

            for (long long value : columnSum) {
                currentSum = max(value, currentSum + value);

                answer = max(answer, currentSum);
            }
        }
    }

    return answer;
}
```

Para matrizes aproximadamente quadradas:

$$O(N^3)$$

Se for melhor escolher a menor dimensao para os pares:

$$O(\min(N, M)^2 \max(N, M))$$

7.6 PD de Intervalos

Usada quando o estado representa um intervalo $[l, r]$ e a transiçao divide o intervalo em partes menores.

Forma geral:

$$dp[l][r] = \min_{l \le k < r} (dp[l][k] + dp[k+1][r] + cost(l, k, r))$$

A ordem de preenchimento deve garantir que intervalos menores sejam calculados antes dos maiores.

Complexidade do template: $O(N^3)$.

```
long long intervalDP(const vector<long long>& values) {
    int valueCount = values.size();

    vector<vector<long long>> dp(valueCount, vector<long long>(valueCount, 0));

    for (int length = 2; length <= valueCount; length++) {
        for (int left = 0; left + length - 1 < valueCount; left++) {
            int right = left + length - 1;

            dp[left][right] = INF;

            for (int split = left; split < right; split++) {
                // Adapte o custo conforme o problema.
                long long cost = 0;

                dp[left][right] =
                    min(dp[left][right],
                        dp[left][split] + dp[split + 1][right] + cost);
```

```
            }
        }
    }

    return dp[0][valueCount - 1];
}
```

Padrão de preenchimento:

intervalos pequenos → intervalos grandes

Frequentemente o custo depende de uma soma prefixada:

```
long long intervalSum(const vector<long long>& prefix, int left, int right) {
    return prefix[right + 1] - prefix[left];
}
```